

Use Cases: Generative AI in Code Generation

Generative AI (GenAI) is reshaping modern software engineering by acting as an **intelligent coding assistant** that accelerates development, reduces operational costs, and boosts developer productivity. Two of the most impactful use cases are:

1. **Saving Time and Cost**
 2. **Maximizing Software Development Productivity**
-

1. Use Case: Utilizing Generative AI to Save Time and Cost

Business Challenge

Organizations face:

- Long development cycles
- High engineering costs
- Repetitive coding tasks
- Large maintenance overhead

Much developer time is spent on **boilerplate code**, not core innovation.

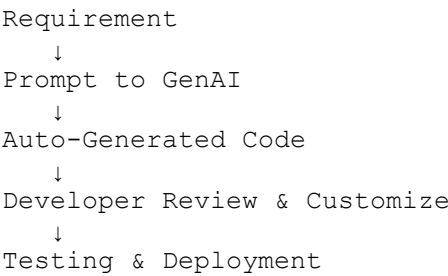
How Generative AI Solves This

GenAI can automatically:

- Generate project scaffolding
- Create CRUD APIs
- Produce UI components
- Generate documentation and tests

This reduces **manual effort** and **engineering hours**.

Typical Workflow



Example: Backend API Generation

Prompt

Generate a Flask REST API for managing students with create and list operations.

Generated Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)
students = []

@app.route("/students", methods=["POST"])
def add_student():
    students.append(request.json)
    return {"message": "Student added"}

@app.route("/students", methods=["GET"])
def get_students():
    return jsonify(students)

if __name__ == "__main__":
    app.run(debug=True)
```

Time & Cost Impact

Task	Traditional Effort	With GenAI
API creation	3–4 hours	15–20 mins
Documentation	1 hour	5 mins
Unit tests	2 hours	20 mins

Result:
Up to **60–70% reduction** in development time.

Business Value

- ✓ Faster time-to-market
 - ✓ Lower staffing cost
 - ✓ More features per sprint
 - ✓ Improved ROI
-

2. Use Case: Maximizing Software Development Productivity with Generative AI

Productivity Problem

Developers often spend time on:

- Searching solutions
- Writing repetitive logic
- Fixing simple bugs

Instead of focusing on **business logic & architecture**.

How GenAI Boosts Productivity

- Smart code completion
 - Debugging suggestions
 - Refactoring support
 - Auto test generation
-

Example: Bug Fix Assistance

Original Code

```
def average(nums):  
    return sum(nums)/len(nums)
```

Prompt

```
Fix potential issues
```

AI Output

```
def average(nums):  
    if not nums:  
        return 0  
    return sum(nums)/len(nums)
```

Example: Feature Enhancement

Prompt

Add input validation and error handling

AI Output

```
if not request.json:  
    return {"error": "Invalid input"}, 400
```

Productivity Gains

Area	Without GenAI	With GenAI
Coding speed	Medium	High
Debugging time	High	Low
Learning curve	Slow	Faster

Team-Level Impact

- ✓ Developers focus on logic
 - ✓ Higher-quality code
 - ✓ Reduced burnout
 - ✓ Faster delivery
-

3. Key Takeaways

- GenAI **saves time and cost** by automating repetitive work
 - GenAI **maximizes productivity** by acting as an AI pair programmer
 - Human review remains essential
-

4. Quick Check

Q1. Main benefit of GenAI in saving cost?

- A) Removing developers
- B) Automating repetitive coding
- C) Eliminating testing
- D) Increasing servers

✓ Answer: B

Q2. True or False

GenAI can generate documentation and tests.

✓ True

Q3. Short Answer

How does GenAI improve productivity?

✓ By assisting with coding, debugging, and refactoring.

Q4. Scenario

Your startup wants MVP quickly. What should you do?

✓ Use GenAI to scaffold backend & frontend.

Summary

Generative AI enables organizations to build software **faster, cheaper, and smarter**. When integrated properly, it becomes a powerful productivity engine rather than just a coding tool.
